

C++ Programming

Day 8: 상속과 다형성

상속, 접근 지정자, 오버라이딩, 가상 함수, 추상 클래스, 업캐스팅

Contents

1	학습 목표	2
2	상속	2
2.1	상속 문법	3
2.2	부모 생성자 호출	3
2.3	전체 예제	3
3	접근 지정자와 상속	4
3.1	public	4
3.2	private	5
3.3	protected	5
3.4	public 상속의 의미	5
3.5	전체 예제	6
4	함수 오버라이딩	7
4.1	부모 함수 직접 호출	7
4.2	오버로딩과 오버라이딩	7
4.3	전체 예제	8
5	가상 함수와 동적 바인딩	8
5.1	virtual 함수	9
5.2	override 키워드	9
5.3	포인터와 참조에서의 다형성	10
5.4	전체 예제	10
6	순수 가상 함수와 추상 클래스	11
6.1	추상 클래스	11
6.2	자식의 구현 의무	11
6.3	가상 함수와 순수 가상 함수 비교	11
6.4	전체 예제	12
7	업캐스팅과 다형성	12
7.1	업캐스팅을 사용하는 이유	12
7.2	가상 소멸자	13
7.3	= default의 의미	13
7.4	전체 예제	14
8	종합 정리	15
9	종합 실습	15
9.1	실습 완성 코드	15
10	실습 파일 구성	17

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 기본 클래스와 파생 클래스의 관계 설명하기
- 초기화 리스트를 사용하여 부모 생성자 호출하기
- `public`, `protected`, `private`의 접근 범위 구분하기
- 부모 함수를 자식 클래스에서 오버라이딩하기
- 정적 바인딩과 동적 바인딩의 차이 설명하기
- `virtual`과 `override`를 사용하여 런타임 다형성 구현하기
- 순수 가상 함수와 추상 클래스의 역할 이해하기
- 업캐스팅을 이용하여 여러 자식 객체를 부모 포인터로 처리하기
- 다형적 부모 클래스에서 가상 소멸자가 필요한 이유 설명하기
- `= default`가 컴파일러의 기본 구현을 요청하는 문법임을 이해하기

2. 상속

상속(inheritance)은 기존 클래스를 기반으로 새로운 클래스를 만드는 기능이다. 여러 클래스가 공통 데이터와 기능을 가질 때, 공통 부분을 부모 클래스에 작성하고 자식 클래스에서 재사용할 수 있다.

예를 들어 학생과 교수는 모두 이름을 가진 사람이다. 상속을 사용하지 않으면 각 클래스에 같은 멤버를 반복해서 작성해야 한다.

```

1 class Student
2 {
3 public:
4     string name;
5 };
6
7 class Professor
8 {
9 public:
10    string name;
11 };

```

공통 멤버를 `Person`에 모으면 중복을 줄일 수 있다.

```

1 class Person
2 {
3 public:
4     string name;
5 };
6
7 class Student : public Person
8 {
9 };

```

`Person`은 기본 클래스(base class), 부모 클래스(parent class)라고 부른다. `Student`는 파생 클래스(derived class), 자식 클래스(child class)라고 부른다.

2.1. 상속 문법

```
1 class Child : public Parent
2 {
3 };
```

위 코드는 Child가 Parent를 public 방식으로 상속한다는 뜻이다. 자식 객체 안에는 개념적으로 부모 클래스 부분과 자식 클래스 부분이 함께 존재한다.

Student object

```
+-----+
| Person part |
| name       |
+-----+
| Student part |
| major      |
+-----+
```

2.2. 부모 생성자 호출

생성자는 그대로 상속되지 않는다. 자식 객체를 만들 때는 부모 부분이 먼저 초기화되고, 그 다음 자식 부분이 초기화된다.

1. Parent constructor
2. Child constructor
3. Child object completed

부모 생성자는 초기화 리스트(initializer list)에서 호출한다.

```
1 Student(string name, string major)
2   : Person(name)
3 {
4   this->major = major;
5 }
```

Person(name)은 이미 존재하는 객체의 멤버에 접근하는 표현이 아니다. Person 생성자를 호출하여 자식 객체 안의 부모 부분을 초기화하는 표현이다. Python의 `super().__init__(name)`과 비슷한 역할을 한다.

핵심 포인트

자식 생성자의 본문이 시작되기 전에 부모 부분이 먼저 만들어진다. 부모 클래스에 기본 생성자가 없다면 초기화 리스트에서 적절한 부모 생성자를 반드시 호출해야 한다.

2.3. 전체 예제

```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 class Person
7 {
8 public:
9     string name;
10
11     Person(string name)
```

```

12     {
13         this->name = name;
14     }
15
16     void introduce()
17     {
18         cout << "Name : " << name << endl;
19     }
20 };
21
22 class Student : public Person
23 {
24 public:
25     string major;
26
27     Student(string name, string major)
28         : Person(name)
29     {
30         this->major = major;
31     }
32
33     void showInfo()
34     {
35         introduce();
36         cout << "Major : " << major << endl;
37     }
38 };
39
40 int main()
41 {
42     Student student("Alice", "Physics");
43     student.showInfo();
44
45     return 0;
46 }

```

출력 결과는 다음과 같다.

```

Name : Alice
Major : Physics

```

3. 접근 지정자와 상속

클래스의 멤버는 public, protected, private 접근 지정자로 외부 접근 범위를 제한할 수 있다.

접근 위치	public	protected	private
같은 클래스 내부	가능	가능	가능
자식 클래스 내부	가능	가능	불가능
외부 코드	가능	불가능	불가능

3.1. public

public 멤버는 클래스 내부, 자식 클래스, 외부 코드에서 모두 접근할 수 있다.

```

1 class Person
2 {
3 public:

```

```

4     string name;
5 };
6
7 Person person;
8 person.name = "Alice";

```

3.2. private

private 멤버는 선언된 클래스 내부에서만 직접 접근할 수 있다. 자식 클래스에서도 직접 접근할 수 없다.

```

1 class Person
2 {
3 private:
4     string name;
5 };

```

외부에서 값을 읽거나 변경해야 한다면 public 멤버 함수를 제공한다.

```

1 class Person
2 {
3 private:
4     string name;
5
6 public:
7     void setName(string name)
8     {
9         this->name = name;
10    }
11 };

```

3.3. protected

protected 멤버는 외부에서 접근할 수 없지만 자식 클래스에서는 직접 접근할 수 있다.

```

1 class Vehicle
2 {
3 protected:
4     string brand;
5 };
6
7 class Car : public Vehicle
8 {
9 public:
10     void showBrand()
11     {
12         cout << brand << endl;
13     }
14 };

```

Car의 멤버 함수에서는 brand를 사용할 수 있지만, main()에서 car.brand로 접근하면 컴파일 오류가 발생한다.

3.4. public 상속의 의미

다음 코드에서 콜론 뒤의 public은 멤버 접근 지정자가 아니라 상속 방식이다.

```

1 class Car : public Vehicle
2 {
3 };

```

public 상속에서는 부모의 public 멤버가 자식에서도 public으로 유지되고, 부모의 protected 멤버가 자식에서도 protected로 유지된다. 부모의 private 멤버는 자식 객체 안에 존재하지만 자식 클래스가 직접 접근할 수 없다.

부모에서의 접근 권한	public 상속 후 자식에서의 접근 권한
public	public
protected	protected
private	직접 접근 불가능

핵심 포인트

protected는 자식에게 구현 세부 사항을 공개하므로 편리하지만 결합도를 높일 수 있다. 일반적으로 데이터는 private으로 두고 필요한 동작을 멤버 함수로 제공하는 설계가 더 안전하다.

3.5. 전체 예제

```

1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 class Vehicle
7 {
8 protected:
9     string brand;
10
11 public:
12     Vehicle(string brand)
13     {
14         this->brand = brand;
15     }
16 };
17
18 class Car : public Vehicle
19 {
20 private:
21     int year;
22
23 public:
24     Car(string brand, int year)
25         : Vehicle(brand)
26     {
27         this->year = year;
28     }
29
30     void showInfo()
31     {
32         cout << "Brand : " << brand << endl;
33         cout << "Year : " << year << endl;
34     }
35 };
36
37 int main()
38 {
39     Car car("Hyundai", 2025);
40     car.showInfo();
41 }

```

```

42     return 0;
43 }

```

4. 함수 오버라이딩

함수 오버라이딩(function overriding)은 부모 클래스의 멤버 함수를 자식 클래스에서 같은 형태로 다시 정의하는 것이다. 부모가 제공하는 기본 동작을 자식의 특성에 맞게 바꿀 때 사용한다.

```

1  class Animal
2  {
3  public:
4      void sound()
5      {
6          cout << "Some sound" << endl;
7      }
8  };
9
10 class Dog : public Animal
11 {
12 public:
13     void sound()
14     {
15         cout << "Bark!" << endl;
16     }
17 };

```

Dog 객체에서 sound()를 호출하면 컴파일러는 가장 가까운 범위인 Dog에서 먼저 함수를 찾는다.

```

1 Dog dog;
2 dog.sound();

```

출력은 Bark!이다.

4.1. 부모 함수 직접 호출

자식 클래스 내부에서 부모 구현을 직접 호출하려면 범위 지정 연산자 ::를 사용한다.

```

1 void sound()
2 {
3     Animal::sound();
4     cout << "Bark!" << endl;
5 }

```

Animal::sound()는 Animal 범위에 있는 sound()를 명시적으로 선택한다. std::cout에서 std::를 사용하는 것과 같은 종류의 연산자이다.

4.2. 오버로딩과 오버라이딩

두 용어는 이름이 비슷하지만 서로 다른 개념이다.

구분	오버로딩	오버라이딩
관계	같은 범위	부모와 자식의 상속 관계
함수 이름	같음	같음
매개변수	달라야 함	기본적으로 같아야 함
목적	여러 입력 형태 처리	부모 동작 재정의

4.3. 전체 예제

```

1 #include <iostream>
2
3 using namespace std;
4
5 class Shape
6 {
7 public:
8     void draw()
9     {
10         cout << "Drawing Shape" << endl;
11     }
12 };
13
14 class Circle : public Shape
15 {
16 public:
17     void draw()
18     {
19         cout << "Drawing Circle" << endl;
20     }
21
22     void showDrawing()
23     {
24         Shape::draw();
25         draw();
26     }
27 };
28
29 int main()
30 {
31     Circle circle;
32     circle.showDrawing();
33
34     return 0;
35 }

```

출력 결과는 다음과 같다.

```

Drawing Shape
Drawing Circle

```

핵심 포인트

같은 이름의 함수를 자식에 다시 정의하는 것만으로는 부모 포인터를 통한 다형성이 완성되지 않는다. 부모 포인터가 실제 자식 객체의 함수를 호출하게 하려면 부모 함수에 `virtual`이 필요하다.

5. 가상 함수와 동적 바인딩

부모 포인터는 자식 객체를 가리킬 수 있다.

```

1 Animal* animal = new Dog();

```

실제로 생성된 객체는 `Dog`이지만 포인터의 정적 타입은 `Animal*`이다. 부모 함수가 가상 함수가 아니라면 함수 호출은 포인터의 정적 타입을 기준으로 결정된다.

```

1 class Animal
2 {
3 public:
4     void sound()
5     {
6         cout << "Some sound" << endl;
7     }
8 };

```

```

1 Animal* animal = new Dog();
2 animal->sound();

```

이 경우 `Animal::sound()`가 호출된다. 컴파일 시점에 호출 대상을 결정하는 방식을 정적 바인딩(static binding)이라고 한다.

5.1. virtual 함수

부모 함수 앞에 `virtual`을 붙이면 실제 객체의 타입을 기준으로 호출 대상을 결정한다.

```

1 class Animal
2 {
3 public:
4     virtual void sound()
5     {
6         cout << "Some sound" << endl;
7     }
8 };

```

이제 같은 호출에서도 `Dog::sound()`가 실행된다.

```

Animal* pointer
    |
    v
Actual object: Dog
    |
    v
Dog::sound()

```

런타임에 실제 객체를 확인하여 함수를 선택하는 방식을 동적 바인딩(dynamic binding)이라고 한다.

5.2. override 키워드

부모에서 가상 함수로 선언된 함수는 자식에서도 가상 함수의 성질을 유지한다. 현대 C++에서는 자식 함수 뒤에 `override`를 붙이는 것이 권장된다.

```

1 class Dog : public Animal
2 {
3 public:
4     void sound() override
5     {
6         cout << "Bark!" << endl;
7     }
8 };

```

`override`는 이 함수가 부모의 가상 함수를 정확히 재정의한다는 의도를 컴파일러에 전달한다. 함수 이름이나 매개변수 형식을 잘못 작성하면 컴파일 오류가 발생하므로 실수를 빠르게 발견할 수 있다.

```

1 void soud() override
2 {
3 }

```

위 코드는 부모에 같은 형태의 가상 함수가 없으므로 컴파일되지 않는다.

5.3. 포인터와 참조에서의 다형성

가상 함수의 동적 바인딩은 부모 타입의 포인터뿐 아니라 부모 타입의 참조에서도 동작한다.

```

1 void makeSound(Animal& animal)
2 {
3     animal.sound();
4 }
5
6 Dog dog;
7 makeSound(dog);

```

객체를 값으로 복사해서 받으면 자식 부분이 잘리는 객체 슬라이싱(object slicing)이 발생할 수 있으므로 다형성을 사용할 때는 부모 포인터 또는 부모 참조를 사용한다.

5.4. 전체 예제

```

1 #include <iostream>
2
3 using namespace std;
4
5 class Shape
6 {
7 public:
8     virtual void draw()
9     {
10         cout << "Drawing Shape" << endl;
11     }
12
13     virtual ~Shape() = default;
14 };
15
16 class Rectangle : public Shape
17 {
18 public:
19     void draw() override
20     {
21         cout << "Drawing Rectangle" << endl;
22     }
23 };
24
25 int main()
26 {
27     Shape* shape = new Rectangle();
28     shape->draw();
29
30     delete shape;
31
32     return 0;
33 }

```

출력 결과는 다음과 같다.

Drawing Rectangle

6. 순수 가상 함수와 추상 클래스

부모 클래스가 공통 동작의 이름만 정하고 구체적인 구현은 자식에게 맡겨야 하는 경우가 있다. 예를 들어 모든 동물은 소리를 낼 수 있지만, 일반적인 `Animal` 자체의 구체적인 소리는 정의하기 어렵다.

이때 순수 가상 함수(pure virtual function)를 사용한다.

```
1 virtual void sound() = 0;
```

끝의 `= 0`은 이 함수가 순수 가상 함수라는 뜻이다.

6.1. 추상 클래스

순수 가상 함수를 하나 이상 가진 클래스는 추상 클래스(abstract class)가 된다. 추상 클래스는 완성되지 않은 인터페이스를 포함하므로 직접 객체를 생성할 수 없다.

```
1 class Animal
2 {
3 public:
4     virtual void sound() = 0;
5 };
6
7 Animal animal;
```

마지막 줄은 컴파일 오류이다. 그러나 추상 클래스의 포인터나 참조는 사용할 수 있다.

```
1 Animal* animal = new Dog();
```

실제로 생성되는 객체가 순수 가상 함수를 구현한 구체 클래스 `Dog`이기 때문이다.

6.2. 자식의 구현 의무

자식 클래스가 모든 순수 가상 함수를 구현하지 않으면 그 자식 클래스도 추상 클래스가 된다.

```
1 class Dog : public Animal
2 {
3 };
```

`Dog`가 `sound()`를 구현하지 않았으므로 `Dog` 객체도 생성할 수 없다.

```
1 class Dog : public Animal
2 {
3 public:
4     void sound() override
5     {
6         cout << "Bark!" << endl;
7     }
8 };
```

이제 `Dog`는 객체를 만들 수 있는 구체 클래스(concrete class)가 된다.

6.3. 가상 함수와 순수 가상 함수 비교

구분	가상 함수	순수 가상 함수
기본 구현	있음	일반적으로 없음
문법	<code>virtual void f()</code>	<code>virtual void f() = 0</code>
부모 객체 생성	가능	불가능
자식의 재정의	선택 가능	구체 클래스가 되려면 필수
주요 역할	기본 동작 제공	공통 인터페이스 강제

6.4. 전체 예제

```

1 #include <iostream>
2
3 using namespace std;
4
5 class Device
6 {
7 public:
8     virtual void start() = 0;
9     virtual ~Device() = default;
10 };
11
12 class Computer : public Device
13 {
14 public:
15     void start() override
16     {
17         cout << "Computer starts" << endl;
18     }
19 };
20
21 int main()
22 {
23     Device* device = new Computer();
24     device->start();
25
26     delete device;
27
28     return 0;
29 }

```

7. 업캐스팅과 다형성

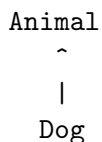
업캐스팅(upcasting)은 자식 객체를 부모 타입의 포인터나 참조로 다루는 것이다.

```

1 Animal* animal = new Dog();

```

상속 계층에서 자식에서 부모 방향으로 타입을 올려서 다루므로 업캐스팅이라고 부른다. 업캐스팅은 안전한 방향의 변환이므로 명시적 형 변환 없이 사용할 수 있다.



7.1. 업캐스팅을 사용하는 이유

업캐스팅을 사용하면 서로 다른 자식 객체를 하나의 부모 타입으로 묶어 처리할 수 있다.

```

1 Animal* animals[3];
2
3 animals[0] = new Dog();
4 animals[1] = new Cat();
5 animals[2] = new Cow();

```

반복문에서는 실제 자식 타입을 따로 확인하지 않고 같은 코드로 함수를 호출할 수 있다.

```

1 for (int i = 0; i < 3; i++)
2 {
3     animals[i]->sound();
4 }

```

호출 표현은 모두 같지만 실제 객체에 따라 서로 다른 함수가 실행된다.

Dog object -> Bark!

Cat object -> Meow!

Cow object -> Moo!

하나의 부모 인터페이스가 여러 형태의 동작을 나타내는 성질을 다형성(polymorphism)이라고 한다.

7.2. 가상 소멸자

부모 포인터로 자식 객체를 삭제할 때는 부모 소멸자가 가상 함수여야 한다.

```

1 Animal* animal = new Dog();
2 delete animal;

```

부모 소멸자가 가상이면 부모 포인터를 통한 삭제에서 자식 소멸자가 호출되지 않을 수 있으며, 이는 정의되지 않은 동작이다. 자식 클래스가 동적 메모리나 파일 같은 자원을 소유한다면 자원이 제대로 해제되지 않을 수 있다.

따라서 다형적으로 사용되는 부모 클래스는 다음과 같이 가상 소멸자를 선언한다.

```

1 virtual ~Animal() = default;

```

소멸자도 가상 함수이므로 실제 객체의 타입을 따라 자식 소멸자부터 호출되고 마지막으로 부모 소멸자가 호출된다.

Dog destructor

Animal destructor

7.3. = default의 의미

= default는 해당 특수 멤버 함수에 대해 컴파일러가 제공하는 기본 구현을 사용하겠다는 뜻이다.

```

1 virtual ~Animal() = default;

```

위 코드는 소멸자를 가상 함수로 만들되 별도의 정리 코드를 직접 작성하지 않고 컴파일러의 기본 소멸 동작을 사용한다는 의미이다.

다음 특수 멤버 함수에도 사용할 수 있다.

```

1 class Example
2 {
3 public:
4     Example() = default;
5     Example(const Example&) = default;
6     Example& operator=(const Example&) = default;
7     ~Example() = default;
8 };

```

직접 관리하는 자원이 있다면 기본 구현으로 충분하지 않을 수 있다. 예를 들어 클래스가 new로 메모리를 할당했다면 소멸자에서 직접 delete를 수행하거나 표준 자원 관리 클래스를 사용해야 한다.

핵심 포인트

= default는 빈 함수 본문을 단순히 줄여 쓴 문법이라는 의미만은 아니다. 컴파일러가 제공하는 기본 특수 멤버 함수의 성질을 그대로 유지하며, 개발자가 그 기본 동작을 의도적으로 선택했음을 명확히 나타낸다.

7.4. 전체 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 class Animal
6 {
7 public:
8     virtual void sound() = 0;
9     virtual ~Animal() = default;
10 };
11
12 class Dog : public Animal
13 {
14 public:
15     void sound() override
16     {
17         cout << "Bark!" << endl;
18     }
19 };
20
21 class Cat : public Animal
22 {
23 public:
24     void sound() override
25     {
26         cout << "Meow!" << endl;
27     }
28 };
29
30 int main()
31 {
32     Animal* animals[2];
33
34     animals[0] = new Dog();
35     animals[1] = new Cat();
36
37     for (int i = 0; i < 2; i++)
38     {
39         animals[i]->sound();
40     }
41
42     for (int i = 0; i < 2; i++)
43     {
44         delete animals[i];
45     }
46
47     return 0;
48 }
```

출력 결과는 다음과 같다.

Bark!

Meow!

8. 종합 정리

핵심 포인트

- 상속은 기존 클래스의 데이터와 기능을 바탕으로 새로운 클래스를 만드는 기능이다.
- 자식 객체를 만들 때 부모 생성자가 먼저 호출되며 초기화 리스트로 부모 생성자를 선택한다.
- `protected` 멤버는 외부에서 접근할 수 없지만 자식 클래스에서는 접근할 수 있다.
- 오버라이딩은 부모의 멤버 함수를 자식에서 같은 형태로 다시 정의하는 것이다.
- 비가상 함수는 정적 타입을 기준으로 호출되고, 가상 함수는 실제 객체 타입을 기준으로 호출된다.
- `override`는 부모의 가상 함수를 정확히 재정의했는지 컴파일러가 검사하게 한다.
- 순수 가상 함수를 가진 클래스는 추상 클래스가 되어 직접 객체를 만들 수 없다.
- 업캐스팅은 여러 자식 객체를 하나의 부모 타입으로 묶어 처리할 수 있게 한다.
- 같은 부모 인터페이스가 실제 객체에 따라 다르게 동작하는 성질이 다형성이다.
- 다형적으로 삭제되는 부모 클래스에는 가상 소멸자가 필요하다.
- `= default`는 컴파일러의 기본 특수 멤버 함수 구현을 사용하겠다는 명시적인 선언이다.

9. 종합 실습

실습

Vehicle 계층을 작성한다.

- Vehicle은 추상 클래스이다.
- Vehicle에 순수 가상 함수 `move()`를 선언한다.
- Vehicle에 가상 소멸자를 `= default`로 선언한다.
- Car는 `move()`에서 "Car is moving"을 출력한다.
- Bike는 `move()`에서 "Bike is moving"을 출력한다.
- 부모 포인터 배열에 Car와 Bike 객체를 저장한다.
- 반복문으로 모든 `move()`를 호출한다.
- 반복문으로 모든 동적 객체를 삭제한다.

9.1. 실습 완성 코드

```

1 #include <iostream>
2
3 using namespace std;
4
5 class Vehicle
6 {
7 public:

```



```
8     virtual void move() = 0;
9     virtual ~Vehicle() = default;
10 };
11
12 class Car : public Vehicle
13 {
14 public:
15     void move() override
16     {
17         cout << "Car is moving" << endl;
18     }
19 };
20
21 class Bike : public Vehicle
22 {
23 public:
24     void move() override
25     {
26         cout << "Bike is moving" << endl;
27     }
28 };
29
30 int main()
31 {
32     Vehicle* vehicles[2];
33
34     vehicles[0] = new Car();
35     vehicles[1] = new Bike();
36
37     for (int i = 0; i < 2; i++)
38     {
39         vehicles[i]->move();
40     }
41
42     for (int i = 0; i < 2; i++)
43     {
44         delete vehicles[i];
45     }
46
47     return 0;
48 }
```

10. 실습 파일 구성

실습

Day 8 파일은 다음과 같이 관리한다.

- 01_inheritance.cpp
- 02_access_specifier.cpp
- 03_function_overriding.cpp
- 04_virtual_function.cpp
- 05_abstract_class.cpp
- 06_upcasting.cpp
- practice/p01_inheritance.cpp
- practice/p02_access_specifier.cpp
- practice/p03_function_overriding.cpp
- practice/p04_virtual_function.cpp
- practice/p05_abstract_class.cpp
- practice/p06_upcasting.cpp